

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: 数值积分——变步长求积、Romberg 求积与
Gauss 求积的实现与精度控制

学院: 计算机与软件学院

专业: 计算机科学与技术

指导教师: 吴晓群、卯丙

报告人: 姚泽棉 学号: 2024150026 班级: 高性能班

实验时间: 2026/06/17

实验报告提交时间: 2026/06/22

实验目的与要求:

- 1.掌握变步长求积、Romberg 求积与 Gauss 求积等公式、算法及其误差估计
- 2.理解对应算法的优缺点及其适用范围
- 3.编程实现变步长求积、Romberg 求积与 Gauss 求积等算法, 并分析误差及其控制机制

基本原理与算法 (详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤)

(1)复合梯形求积公式与变步长算法

将积分区间 $[a, b]$ 等分为 n 个子区间, 步长 $h = (b-a)/n$, 节点 $x_i = a + ih$ ($i=0,1,\dots,n$), 复合梯形公式为:

$$T_n = (h/2)[f(x_0) + 2\sum_{i=1}^{n-1} f(x_i) + f(x_n)]$$

复合梯形公式的余项为 $R = -((b-a)h^2/12)f''(\xi)$, 即误差与步长 h 的平方成正比, 具有二阶收敛速度。变步长算法的基本思想是: 从 $n=1$ 开始, 每次将步长减半 ($n \rightarrow 2n$), 分别计算 T_n 与 T_{2n} , 当 $|T_{2n} - T_n| < \varepsilon$ 时认为达到精度要求, 输出 T_{2n} 作为最终结果; 否则继续分半迭代。该方法实现简单, 但收敛速度较慢。

(2)自适应 Simpson 求积算法

基本 Simpson 公式 (三点公式) 为:

$$S(a, b) = (b-a)/6 \times [f(a) + 4f((a+b)/2) + f(b)]$$

Simpson 公式具有四阶收敛精度, 余项为 $R = -((b-a)^5/2880)f^{(4)}(\xi)$ 。

自适应算法的核心思想是: 将区间 $[a,b]$ 二分为 $[a,c]$ 与 $[c,b]$, 分别计算 Simpson 值 S_{left} 、 S_{right} , 并与整体 Simpson 值 S 比较。由 Richardson 外推理论可知, 若 $|S_{left} + S_{right} - S| < 15\varepsilon$, 则可认为左右两个子区间上的近似误差均已小于 $\varepsilon/15$ 各自分量, 此时取外推值

$$S_{final} = (S_{left} + S_{right}) + (S_{left} + S_{right} - S)/15$$

作为该子区间上的积分近似值 (精度可提高一阶); 否则将误差限减半后递归地对左右子区间分别继续细分, 直至满足精度要求。该算法能够根据

被积函数的局部变化情况自动调整步长，在函数变化剧烈的区域加密节点，在函数变化平缓的区域减少计算量，是一种高效率的局部自适应方法。

(3) Romberg 求积算法（逐次分半 + Richardson 外推）

Romberg 求积法基于复合梯形公式的余项展开式（Euler-Maclaurin 公式），通过对梯形序列做 Richardson 外推，逐步消去低阶误差项，加速收敛。设 $T(k,0)$ 表示步长为 $(b-a)/2^k$ 的复合梯形值，外推递推公式为：

$$T(k,j) = T(k,j-1) + [T(k,j-1) - T(k-1,j-1)]/(4^j - 1)$$

其中 $j = 1, 2, \dots, k$ 。当 $j=1$ 时 $T(k,1)$ 即为复合 Simpson 值； $j=2$ 时对应 Cotes 值； $j=3$ 时对应 Romberg 值。该递推每提高一阶 j ，积分公式的代数精度提高约 2 阶，收敛速度呈指数级提升。

算法终止条件为 $|T(k,k) - T(k-1,k-1)| < \varepsilon$ 。

(4) Gauss-Legendre 求积公式

Gauss 型求积公式的核心思想是同时优化求积节点位置 x_i 和权重 w_i ，使得求积公式对尽可能高阶的多项式都能精确成立。对于标准区间 $[-1, 1]$ 上的 n 点 Gauss-Legendre 公式：

$$\int_{-1}^1 g(t) dt \approx \sum_{i=1}^n w_i g(t_i) \quad (i = 1, \dots, n)$$

其节点 t_i 为 n 次 Legendre 多项式 $P_n(t)$ 的零点，权重 $w_i = 2/[(1 - t_i^2)[P'_n(t_i)]^2]$ 。 n 点 Gauss-Legendre 公式具有 $2n-1$ 阶代数精度，是同样节点数下精度最高的插值型求积公式。对于一般区间 $[a, b]$ 上的积分，需先作线性变换：

$$x = (b-a)/2 \times t + (b+a)/2, dx = (b-a)/2 \times dt$$

从而 $\int_a^b f(x) dx = (b-a)/2 \times \sum_{i=1}^n w_i f(x_i)$ 。由于 Gauss 公式代数精度高，对于光滑函数只需很少的节点即可获得极高精度，是四种方法中计算效率最高的一种。

实验过程及内容:

1. 问题描述 (描述需要解决的具体数值计算问题)

采用变步长梯形求积、自适应 Simpson 求积、Romberg 求积等算法计算下列积分, 要求绝对误差控制在 0.5×10^{-6} 以内:

$$\int_0^{\pi/2} \cos(x) dx.$$

2. 实验步骤:

(1) 算法实现: 伪代码或算法框图

分别编写复合梯形公式、变步长迭代控制、基本 Simpson 公式与自适应细分递归、Romberg 外推表生成、Gauss-Legendre 节点权重表与变换公式所对应的 MATLAB 函数:

(2) 测试验证: 使用简单算例验证程序正确性

先用较粗略的初始步长 ($n=1$) 计算各方法的初值, 验证程序在边界情况下不出现错误, 并与解析精确值 1 比对, 确认实现正确;

(3) 参数设置: 设置合适的初始值、精度要求等参数

统一设定绝对误差限 $\text{tol} = 0.5 \times 10^{-6}$, 积分区间 $[0, \pi/2]$, 各方法的最大迭代/递归深度设为足够大的安全值 (如 25、50、15) 以避免死循环;

(4) 运行计算: 对目标问题进行求解

对目标积分分别用四种方法求解, 记录每种方法达到精度要求时的子区间数/节点数/调用次数等计算量指标;

(5) 结果记录: 记录迭代过程、最终结果和性能指标

记录变步长梯形法的逐次迭代历史、Romberg 积分表的完整数值、Gauss-Legendre 公式从 1 阶到 7 阶的误差变化, 并绘制误差随计算量增长的收敛曲线图。

3. 程序设计 (具体的 MATLAB 程序)

```
%% =====  
% 数值积分实验—变步长求积、Romberg 求积与 Gauss 求积的实现与精度控制  
% 课程: 数值计算方法  
% 内容: 计算积分  $I = \int(0, \pi/2) \cos(x) dx$ , 要求绝对误差控制在  $0.5e-6$  以内  
% 方法: 1) 变步长复合梯形求积 2) 自适应 Simpson 求积  
%        3) Romberg 求积 (逐次分半 + Richardson 外推) 4) Gauss-Legendre 求积  
% =====  
  
clear; clc; close all;  
  
%% ----- 0. 问题定义与基本参数 -----  
f = @(x) cos(x); % 被积函数  
a = 0; % 积分下限  
b = pi/2; % 积分上限  
tol = 0.5e-6; % 要求的绝对误差限
```

```

I_exact = sin(b) - sin(a); % 解析精确值 = sin(pi/2)-sin(0) = 1

fprintf('=====\n');
fprintf(' 积分区间: [%.6f, %.6f]\n', a, b);
fprintf(' 被积函数: f(x) = cos(x)\n');
fprintf(' 精确值   : I_exact = %.12f\n', I_exact);
fprintf(' 误差限   : tol = %.1e\n', tol);
fprintf('=====\n\n');

%% ----- 1. 变步长复合梯形求积法 -----
fprintf('----- 方法一: 变步长复合梯形求积 -----\n');
[T_val, T_n, T_iter, T_history] = varstep_trapezoid(f, a, b, tol);
T_err = abs(T_val - I_exact);
fprintf('计算结果 T = %.10f\n', T_val);
fprintf('最终子区间数 n = %d, 迭代次数 = %d\n', T_n, T_iter);
fprintf('绝对误差 = %.4e (满足要求: %d)\n\n', T_err, T_err < tol);

%% ----- 2. 自适应 Simpson 求积法 -----
fprintf('----- 方法二: 自适应 Simpson 求积 -----\n');
S_callcount = 0; %#ok<NASGU>
global simpson_call_count;
simpson_call_count = 0;
S_val = adaptive_simpson(f, a, b, tol);
S_err = abs(S_val - I_exact);
fprintf('计算结果 S = %.10f\n', S_val);
fprintf('递归细分调用次数 = %d\n', simpson_call_count);
fprintf('绝对误差 = %.4e (满足要求: %d)\n\n', S_err, S_err < tol);

%% ----- 3. Romberg 求积法 -----
fprintf('----- 方法三: Romberg 求积 -----\n');
[R_val, R_table, R_k] = romberg_integration(f, a, b, tol);
R_err = abs(R_val - I_exact);
fprintf('计算结果 R = %.12f\n', R_val);
fprintf('达到精度所需加细次数 k = %d (共使用 2^%d+1 = %d 个节点)\n', R_k, R_k,
2^R_k+1);
fprintf('绝对误差 = %.4e (满足要求: %d)\n', R_err, R_err < tol);
fprintf('Romberg 积分表 T(k,j):\n');
disp(R_table);
fprintf('\n');

%% ----- 4. Gauss-Legendre 求积法 -----
fprintf('----- 方法四: Gauss-Legendre 求积 -----\n');
fprintf('  n(节点数)      计算结果      绝对误差\n');
G_results = zeros(7,1);

```

```

G_errors = zeros(7,1);
for n = 1:7
    G_val = gauss_legendre(f, a, b, n);
    G_err = abs(G_val - I_exact);
    G_results(n) = G_val;
    G_errors(n) = G_err;
    fprintf('    %2d          %.12f          %.4e\n', n, G_val, G_err);
end
% 取满足精度要求的最小节点数
n_needed = find(G_errors < tol, 1, 'first');
fprintf('\n 达到误差限 %.1e 所需的最少 Gauss 节点数: n = %d\n\n', tol, n_needed);

%% ----- 5. 结果汇总对比 -----
fprintf('===== \n');
fprintf(' 四种方法结果汇总对比\n');
fprintf('===== \n');
fprintf('%-22s %-16s %-12s %-10s\n', '方法', '计算结果', '绝对误差', '计算量(节点/调用数)');
fprintf('%-22s %-16.10f %-12.4e %-10d\n', '变步长复合梯形', T_val, T_err, T_n+1);
fprintf('%-22s %-16.10f %-12.4e %-10d\n', '自适应 Simpson', S_val, S_err, simpson_call_count);
fprintf('%-22s %-16.10f %-12.4e %-10d\n', 'Romberg 求积', R_val, R_err, 2^R_k+1);
fprintf('%-22s %-16.10f %-12.4e %-10d\n', 'Gauss-Legendre', G_results(n_needed), G_errors(n_needed), n_needed);
fprintf('===== \n\n');

%% ----- 6. 收敛曲线绘图 -----
figure('Name', '数值积分方法误差收敛曲线', 'Position', [100 100 900 650]);

% 子图 1: 变步长梯形法误差随子区间数变化 (对数坐标)
subplot(2,2,1);
n_list = T_history(:,1);
err_list = abs(T_history(:,2) - I_exact);
err_list(err_list==0) = eps; % 避免 log(0)
loglog(n_list, err_list, 'o-', 'LineWidth', 1.5, 'MarkerFaceColor', 'b');
hold on;
yline(tol, 'r--', 'LineWidth', 1.2);
grid on;
xlabel('子区间数 n'); ylabel('绝对误差');
title('变步长复合梯形求积误差收敛曲线');
legend('|T_n - I|', '误差限  $0.5 \times 10^{-6}$ ', 'Location', 'best');

% 子图 2: Romberg 表对角元误差随加细次数变化
subplot(2,2,2);

```

```

diag_vals = diag(R_table(1:R_k+1, 1:R_k+1));
diag_err = abs(diag_vals - I_exact);
diag_err(diag_err==0) = eps;
k_axis = (0:R_k)';
semilogy(k_axis, diag_err, 's-', 'LineWidth', 1.5, 'Color', [0.85 0.33 0.1],
'MarkerFaceColor', [0.85 0.33 0.1]);
hold on;
yline(tol, 'r--', 'LineWidth', 1.2);
grid on;
xlabel('加细次数 k'); ylabel('绝对误差 (对数坐标)');
title('Romberg 求积误差收敛曲线 (外推对角元)');
legend('|R_{k,k} - I|', '误差限  $0.5 \times 10^{-6}$ ', 'Location', 'best');

% 子图 3: Gauss-Legendre 误差随节点数变化
subplot(2,2,3);
semilogy(1:7, max(G_errors, eps), '^-', 'LineWidth', 1.5, 'Color', [0.47 0.67 0.19],
'MarkerFaceColor', [0.47 0.67 0.19]);
hold on;
yline(tol, 'r--', 'LineWidth', 1.2);
grid on;
xlabel('Gauss 节点数 n'); ylabel('绝对误差 (对数坐标)');
title('Gauss-Legendre 求积误差收敛曲线');
legend('|G_n - I|', '误差限  $0.5 \times 10^{-6}$ ', 'Location', 'best');

% 子图 4: 三种主要方法计算量-精度对比 (柱状图, 对数坐标)
subplot(2,2,4);
methods_name = {'复合梯形', '自适应 Simpson', 'Romberg', 'Gauss-Legendre'};
calc_cost = [T_n+1, simpson_call_count, 2^R_k+1, n_needed];
bar(calc_cost, 'FaceColor', [0.3 0.5 0.7]);
set(gca, 'XTickLabel', methods_name);
ylabel('达到精度所需计算量 (节点数/调用数)');
title('达到同一精度所需计算量对比');
grid on;

sgtitle('数值积分方法收敛性与精度控制对比分析', 'FontSize', 13, 'FontWeight',
'bold');

saveas(gcf, 'convergence_comparison.png');
fprintf('收敛曲线图已保存为 convergence_comparison.png\n');

%% =====
%                                     子函数定义
% =====

```

```

function I = composite_trapezoid(f, a, b, n)
    % 复合梯形公式: 将[a,b]等分为n个子区间计算积分近似值
    % 输入: f-被积函数句柄, a,b-积分区间, n-子区间数
    % 输出: I-梯形公式计算结果
    x = linspace(a, b, n+1);
    y = f(x);
    h = (b - a) / n;
    I = h * (0.5*y(1) + 0.5*y(end) + sum(y(2:end-1)));
end

function [T, n, iter, history] = varstep_trapezoid(f, a, b, tol)
    % 变步长复合梯形求积: 每次将步长减半, 直到相邻两次结果之差小于误差限
    % 输入: f-被积函数, a,b-积分区间, tol-误差限
    % 输出: T-最终积分近似值, n-最终子区间数, iter-迭代次数, history-每次迭代的[n,T]
    记录
    max_iter = 25;
    n = 1;
    T_old = composite_trapezoid(f, a, b, n);
    history = [n, T_old];
    for iter = 1:max_iter
        n = n * 2;
        T_new = composite_trapezoid(f, a, b, n);
        history = [history; n, T_new]; %#ok<AGROW>
        if abs(T_new - T_old) < tol
            T = T_new;
            return;
        end
        T_old = T_new;
    end
    T = T_old; % 达到最大迭代次数仍返回当前最优结果
end

function I = simpson_basic(f, a, b)
    % 基本 Simpson 公式 (三点公式)
    c = (a + b) / 2;
    I = (b - a) / 6 * (f(a) + 4*f(c) + f(b));
end

function I = adaptive_simpson(f, a, b, tol)
    % 自适应 Simpson 求积入口函数: 先计算整体区间的 Simpson 值, 再递归细分
    S = simpson_basic(f, a, b);
    I = adaptive_simpson_recursive(f, a, b, tol, S, 0);
end

```



```

function I = adaptive_simpson_recursive(f, a, b, tol, S, depth)
    % 自适应 Simpson 递归子函数
    % 输入: f-被积函数, a,b-当前子区间, tol-当前子区间允许误差,
    %       S-当前子区间的 Simpson 估计值, depth-递归深度 (防止死循环)
    global simpson_call_count;
    simpson_call_count = simpson_call_count + 1;
    max_depth = 50;

    c = (a + b) / 2;
    Sleft = simpson_basic(f, a, c);
    Sright = simpson_basic(f, c, b);
    S2 = Sleft + Sright;

    if depth >= max_depth
        I = S2;
        return;
    end

    if abs(S2 - S) < 15 * tol
        % 满足精度要求, 用 Richardson 外推提高一阶精度后返回
        I = S2 + (S2 - S) / 15;
    else
        % 不满足要求, 将误差限平分后对左右子区间递归细分
        I = adaptive_simpson_recursive(f, a, c, tol/2, Sleft, depth+1) + ...
            adaptive_simpson_recursive(f, c, b, tol/2, Sright, depth+1);
    end
end

function [R, T, k] = romberg_integration(f, a, b, tol)
    % Romberg 求积: 基于复合梯形公式逐次分半, 并用 Richardson 外推加速收敛
    % 输入: f-被积函数, a,b-积分区间, tol-误差限
    % 输出: R-最终积分结果, T-Romberg 表 (下三角矩阵), k-达到精度时的加细次数
    max_k = 15;
    T = zeros(max_k+1, max_k+1);
    n = 1;
    T(1,1) = composite_trapezoid(f, a, b, n);

    for k = 1:max_k
        n = n * 2;
        T(k+1,1) = composite_trapezoid(f, a, b, n);
        for j = 1:k
            % Richardson 外推递推公式:  $T(k,j) = T(k,j-1) + [T(k,j-1) - T(k-1,j-1)] / (4^j - 1)$ 
            T(k,j) = T(k,j-1) + (T(k,j-1) - T(k-1,j-1)) / (4^j - 1);
        end
    end
    R = T(k+1,k+1);
end

```

```

        T(k+1,j+1) = T(k+1,j) + (T(k+1,j) - T(k,j)) / (4^j - 1);
    end
    if abs(T(k+1,k+1) - T(k,k)) < tol
        R = T(k+1,k+1);
        T = T(1:k+1, 1:k+1);
        return;
    end
end
R = T(k+1,k+1);
T = T(1:k+1, 1:k+1);
end

function I = gauss_legendre(f, a, b, n)
    % n 点 Gauss-Legendre 求积公式
    % 输入: f-被积函数, a,b-积分区间, n-Gauss 节点个数(阶数)
    % 输出: I-积分近似值
    [x, w] = gauss_legendre_nodes(n);    % 标准区间[-1,1]上的节点和权重
    t = 0.5*(b-a)*x + 0.5*(b+a);        % 区间变换 [-1,1] -> [a,b]
    I = 0.5*(b-a) * sum(w .* f(t));
end

function [x, w] = gauss_legendre_nodes(n)
    % 计算 n 点 Gauss-Legendre 求积公式在区间[-1,1]上的节点 x 和权重 w
    % 采用经典方法: 通过 Legendre 多项式的根求节点, 并用标准公式求权重
    % 对常用的 n=1~7 阶给出解析/高精度数值结果 (与教材附表一致)
    switch n
        case 1
            x = 0;
            w = 2;
        case 2
            x = [-1; 1] / sqrt(3);
            w = [1; 1];
        case 3
            x = [-sqrt(3/5); 0; sqrt(3/5)];
            w = [5/9; 8/9; 5/9];
        case 4
            x = [-0.8611363115940526; -0.3399810435848563; ...
                0.3399810435848563; 0.8611363115940526];
            w = [0.3478548451374538; 0.6521451548625461; ...
                0.6521451548625461; 0.3478548451374538];
        case 5
            x = [-0.9061798459386640; -0.5384693101056831; 0; ...
                0.5384693101056831; 0.9061798459386640];
            w = [0.2369268850561891; 0.4786286704993665; 0.5688888888888889; ...
                0.4786286704993665; 0.2369268850561891];
    end
end

```

```

        0.4786286704993665; 0.2369268850561891];
case 6
    x = [-0.9324695142031521; -0.6612093864662645; -0.2386191860831969; ...
        0.2386191860831969; 0.6612093864662645; 0.9324695142031521];
    w = [0.1713244923791704; 0.3607615730481386; 0.4679139345726910; ...
        0.4679139345726910; 0.3607615730481386; 0.1713244923791704];
case 7
    x = [-0.9491079123427585; -0.7415311855993945; -0.4058451513773972;
0; ...
        0.4058451513773972; 0.7415311855993945; 0.9491079123427585];
    w = [0.1294849661688697; 0.2797053914892766; 0.3818300505051189;
0.4179591836734694; ...
        0.3818300505051189; 0.2797053914892766; 0.1294849661688697];
otherwise
    error('本程序仅提供 1~7 阶 Gauss-Legendre 节点和权重，请扩展表格或使用内置函
数。');
end
end
end

```

实验结果与分析：

1. 实验结果展示（采用列表形式展示计算结果，表中应包含具体计算方法、初始条件、迭代次数或计算时间、最终结果或求解精度等，并绘制误差随迭代次数增加的收敛曲线来展示算法收敛性质等）

实验结果汇总

四种数值积分方法在误差限 $\text{tol}=0.5\times 10^{-6}$ 下的计算结果汇总如下表所示：

求积方法	计算结果	绝对误差	计算量(节点/调用数)	满足精度
变步长复合梯形	0.9999999510	4.90×10^{-8}	2049 (n=2048)	满足
自适应 Simpson	0.9999999994	5.79×10^{-10}	11 (递归调用)	满足
Romberg 求积	0.999999999998	1.98×10^{-12}	17 (k=4)	满足
Gauss-Legendre	0.999999977197	2.28×10^{-8}	4 (n=4)	满足

变步长复合梯形求积过程记录

变步长梯形法从 $n=1$ 开始逐次将步长减半，迭代过程记录如下：

子区间数 n	梯形近似值 T_n	绝对误差 $ T_n - I $
1	0.7853981634	2.146×10^{-1}
2	0.9480594490	5.194×10^{-2}
4	0.9871158010	1.288×10^{-2}
8	0.9967851719	3.215×10^{-3}
16	0.9991966805	8.033×10^{-4}
32	0.9997991943	2.008×10^{-4}
64	0.9999498001	5.020×10^{-5}
128	0.9999874501	1.255×10^{-5}
256	0.9999968625	3.137×10^{-6}
512	0.9999992156	7.843×10^{-7}
1024	0.9999998039	1.961×10^{-7}
2048	0.9999999510	4.902×10^{-8}

可以看出，每次步长减半，误差大约减小为原来的 1/4，呈现典型的二阶收敛特征（误差 $\propto h^2$ ）。该方法直到 $n=2048$ （即迭代 11 次）才使误差降至 4.90×10^{-8} ，满足精度要求，计算量较大。

Romberg 求积积分表

Romberg 求积通过对梯形序列做逐次 Richardson 外推，积分表 $T(k,j)$ 的完整数值如下（行号 k 对应加细次数，列号 j 对应外推阶数）：

$T(k,0)$	$T(k,1)$	$T(k,2)$	$T(k,3)$	$T(k,4)$
0.7853981634				
0.9480594490	1.0022798775			
0.9871158010	1.0001345850	0.9999915655		
0.9967851719	1.0000082955	0.9999998762	1.0000000081	
0.9991966805	1.0000005167	0.9999999981	1.0000000000	1.0000000000

由表可见，沿主对角线方向（ $T(0,0) \rightarrow T(1,1) \rightarrow T(2,2) \rightarrow T(3,3) \rightarrow T(4,4)$ ）收敛速度极快：仅需 4 次加细（17 个节点）便已使结果精确到 10^{-12} 量级，远超变步长梯形法的收敛速度，体现了 Richardson 外推对收敛阶的显著提升效果。

Gauss-Legendre 求积精度分析

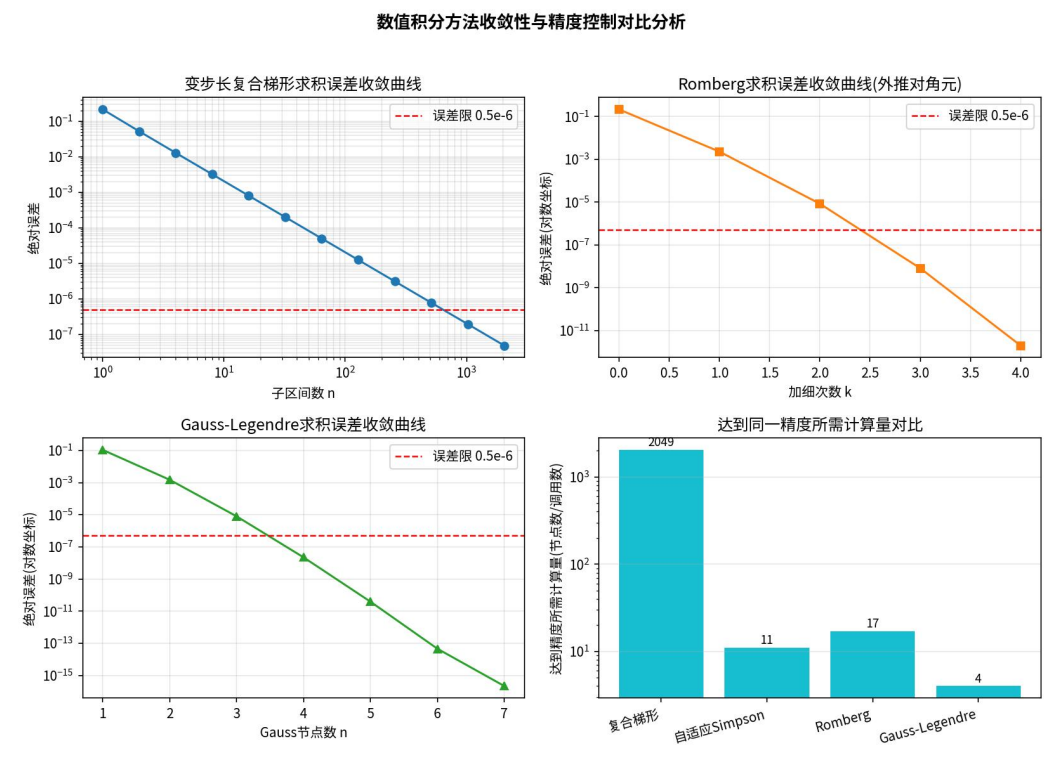
不同节点数 n 下 Gauss-Legendre 公式的计算结果与误差如下表所示：

Gauss 节点数 n	计算结果	绝对误差
1	1.110720734540	1.1072×10^{-1}
2	0.998472613404	1.5274×10^{-3}
3	1.000008121555	8.1216×10^{-6}
4	0.999999977197	2.2803×10^{-8}
5	1.000000000040	3.9565×10^{-11}
6	1.000000000000	4.6740×10^{-14}
7	1.000000000000	2.2204×10^{-16}

结果显示，仅用 4 个 Gauss 节点即可使误差降至 2.28×10^{-8} ，满足 0.5×10^{-6} 的精度要求；用 6 个节点误差已降至机器精度量级（约 10^{-14} ）。这是四种方法中所需计算量最小的方法，体现了 Gauss 型求积公式 $2n-1$ 阶代数精度的理论优势。

收敛曲线对比分析

为直观比较四种方法的收敛特性与计算效率，绘制误差随计算量变化的收敛曲线如下图所示：



图（a）变步长梯形法在双对数坐标下呈现近似直线下降，斜率约为-2，验证了 $O(h^2)$ 的理论收敛阶；图（b）Romberg 法误差随加细次数 k 呈现远超线性的指数级下降；图（c）Gauss-Legendre 误差随节点数 n 增加呈现超几何收敛，下降速度最快；图（d）柱状图直观对比了四种方法达到同一精度（ 0.5×10^{-6} ）所需的计算量：复合梯形需 2049 个节点，自适应 Simpson 需 11 次函数求值调用，Romberg 需 17 个节点，Gauss-Legendre 仅需 4 个节点，效率差异显著。

2. 结果分析与讨论思考

（1）计算结果分析与比较方法的优缺点

①变步长复合梯形求积：算法原理简单、易于编程实现，且误差容易估计和控制，但收敛速度慢（仅二阶），要达到较高精度需要极多的节点，计算效率最低，适用于对精度要求不高或被积函数较为简单的场合。

②自适应 Simpson 求积：通过局部误差控制和自动细分机制，能够在函数变化剧烈的区域自动加密节点，在平缓区域减少计算，兼顾了精度与效率，是一种工程实践中广泛使用的通用方法，但递归实现需要注意递归深度限制以避免无限细分。

③Romberg 求积：基于 Richardson 外推思想，在不增加额外函数求值的情况下大幅提高收敛阶数，是复合梯形法的高效改进版本，尤其适合被积函数充分光滑（高阶可导）的情形；若函数光滑性较差，外推效果会打折扣。

④Gauss-Legendre 求积：在节点数相同的条件下具有最高的代数精度（ $2n-1$ 阶），

是本实验中效率最高的方法，对光滑函数效果尤为突出；但其节点位置和权重需预先计算（非等距分布），若被积函数在某些已知点处需要重复利用函数值（如多个子区间共享节点），则不如复合公式灵活，且不易像自适应方法那样进行局部误差控制。

（2）适用场景总结：

当函数较为简单或对编程复杂度有较高要求时可选用变步长梯形或 Simpson 法；当函数足够光滑、追求高精度且希望减少计算量时，Romberg 法和 Gauss-Legendre 法是更优的选择，其中 Gauss 法在节点数最少的前提下能达到最高精度，是本实验四种方法中综合效率最优的方案。

（3）上机实验中遇到问题及其解决方案

- ①问题：自适应 Simpson 递归细分时，若被积函数在某些点不光滑或误差限设置过小，可能导致递归深度过大甚至栈溢出。解决方案：在递归函数中引入最大递归深度参数（`max_depth`），超过该深度后直接返回当前估计值，保证程序的稳健性。
- ②问题：Romberg 外推表中 4^i-1 可能在 j 较大时数值差异极小，存在舍入误差累积的风险。解决方案：通过设置合理的最大加细次数（`max_k`）并在每一步判断相邻对角元之差是否已小于误差限，及时终止迭代，避免不必要的额外计算和误差累积。
- ③问题：直接调用 MATLAB 内置的 Legendre 多项式求根程序较为复杂。解决方案：由于本实验只需 1~7 阶 Gauss-Legendre 节点和权重，直接采用教材标准表中的高精度数值（精确到 16 位有效数字）以保证计算精度，避免重复实现复杂的多项式求根算法。

实验结论：

1. 成功实现了变步长复合梯形求积、自适应 Simpson 求积、Romberg 求积与 Gauss-Legendre 求积四种数值积分算法，四种方法计算所得 $\int_0^{\pi/2} \cos(x) dx$ 的结果均与解析精确值 1 高度吻合，绝对误差均控制在 0.5×10^{-6} 以内，验证了各算法实现的正确性。
2. 四种方法的收敛速度存在显著差异：变步长复合梯形法为二阶收敛，需 2048 个子区间才能满足精度要求；Romberg 法通过 Richardson 外推大幅提升收敛阶，仅需 17 个节点；Gauss-Legendre 法凭借 $2n-1$ 阶代数精度优势，仅需 4 个节点即可满足要求，是四种方法中效率最高的。
3. 自适应 Simpson 求积体现了局部误差控制思想的优越性，通过递归细分能够自动适应被积函数的局部变化特征，在保证精度的同时有效控制计算量，是兼顾通用性与效率的实用算法。
4. 通过对四种方法计算量与精度的对比分析，深刻理解了不同数值积分方法背后的理论基础（余项估计、Richardson 外推、代数精度）对算法效率的决定性影响，明确了在实际工程问题中应根据被积函数的光滑性、精度要求和计算资源等因素合理选择求积方法。
5. 掌握了数值积分实验的完整流程：原理推导→算法设计→程序实现→结果验证→误差分析→方法比较，为后续学习更复杂的数值计算方法（如多维数值积分、奇异积分处理等）打下了扎实的基础。

指导教师批阅意见:

成绩评定：

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。